
Universidad de las Fuerzas Armadas ESPE

Departamento: Ciencias de la Computación

Carrera: Ingeniería de Software

Tema: Decorator y Iterator

Tarea académico N 2:

1. Información General

- **Asignatura:** Análisis y Diseño de Software
 - **Apellidos y nombres de los estudiantes:**
Panchez Jacome Darwin Javier
Robalino Curay Cristian Isaak
Proaño Vásconez José Ignacio
 - **NRC:** 23305
 - **Fecha de realización:** 16/06/2025
-

2. Objetivo y Desarrollo

Objetivo:

Desarrollar en Java una aplicación CRUD para gestionar estudiantes (id, nombre, edad) implementando los patrones Decorator e Iterator, donde Decorator permitirá añadir funcionalidades como validación y logging de forma modular, e Iterator facilitará el recorrido flexible de las colecciones de estudiantes bajo diferentes criterios.

Desarrollo:

Requisitos Funcionales

- RF1: La aplicación debe permitir agregar un nuevo estudiante con los datos: id, nombre y edad.
- RF2: El sistema debe mostrar todos los estudiantes registrados.

- RF3: El usuario debe poder consultar un estudiante por su número de id.
- RF4: El sistema debe permitir modificar los datos de un estudiante existente.
- RF5: El sistema debe permitir eliminar un estudiante por su número de id.
- RF6: La aplicación debe estar organizada en tres capas: modelo, repositorio, servicio y presentación.

1. Decorator

Estructura del Proyecto

Decorator/

```

├─ Main.java          -> Clase principal
├─ datos/
│   └─ Model/Estudiante.java -> Modelo de datos
│   └─ Repository/EstudianteRepository.java -> Acceso a datos
├─ logica_negocio/
│   └─ EstudianteServiceBase.java -> Servicio base
│   └─ EstudianteServiceDecorator.java -> Decorador abstracto
│   └─ EstudianteService.java -> Implementación concreta
├─ presentacion/
│   └─ EstudianteUI.java -> Interfaz de usuario

```

Código

Clase Main.java

```

import datos.Repository.EstudianteRepository;
import logica_negocio.*;
import presentacion.EstudianteUI;

public class Main {

```

```

public static void main(String[] args) {

    EstudianteRepository repo = new EstudianteRepository();

    IEstudianteService baseService = new EstudianteServiceBase(repo);
    IEstudianteService finalService = baseService;

    new EstudianteUI(finalService);
}
}

```

Decorador datos (model/estudiante.java)

```

/**
 * Clase que representa el modelo de un estudiante.
 * Contiene los atributos básicos y métodos para acceder y modificar
 * los datos del estudiante.
 * Esta clase es utilizada en las demás capas para transferir
 * información de los estudiantes.
 */
public class Estudiante {

    private int id;

    private String nombre;

    private int edad;

    // Constructor para inicializar los datos del estudiante
    public Estudiante(int id, String nombre, int edad) {

        this.id = id;

        this.nombre = nombre;

        this.edad = edad;

    }

    // Métodos getter y setter para acceder y modificar los
    atributos

    public int getId() { return id; }

    public String getNombre() { return nombre; }

```

```

public int getEdad() { return edad; }

public void setNombre(String nombre) { this.nombre = nombre; }
public void setEdad(int edad) { this.edad = edad; }

// Método para mostrar la información del estudiante como texto
@Override
public String toString() {
    return "Estudiante{id=" + id + ", nombre='" + nombre + "',
edad=" + edad + '}';
}
}

```

Decorador datos (repository/EstudianteRepository.java)

```

import datos.Model.Estudiante;
import java.util.*;

/**
 * Clase que representa el repositorio de estudiantes.
 * Se encarga de almacenar, buscar, actualizar y eliminar
 * estudiantes en memoria.
 * Esta clase corresponde a la capa de acceso a datos de la
 * aplicación.
 */
public class EstudianteRepository {

    // Mapa que almacena los estudiantes usando el id como clave
    private final Map<Integer, Estudiante> estudiantes = new
    HashMap<>();

    // Agrega un estudiante al repositorio
    public void agregar(Estudiante estudiante) {
        estudiantes.put(estudiante.getId(), estudiante);
    }

    // Busca un estudiante por su id

```

```

public Estudiante buscarPorId(int id) {
    return estudiantes.get(id);
}

// Devuelve una lista con todos los estudiantes almacenados
public List<Estudiante> obtenerTodos() {
    return new ArrayList<>(estudiantes.values());
}

// Actualiza la información de un estudiante existente
public void actualizar(Estudiante estudiante) {
    estudiantes.put(estudiante.getId(), estudiante);
}

// Elimina un estudiante por su id
public void eliminar(int id) {
    estudiantes.remove(id);
}
}

```

Decorador logica_negocio (EstudianteServiceBase.java)

```

import datos.Model.Estudiante;
import datos.Repository.EstudianteRepository;

import java.util.List;

/**
 * Clase base que implementa la lógica de negocio para el manejo de
 * estudiantes.
 * Implementa la interfaz IEstudianteService.
 */
public class EstudianteServiceBase implements IEstudianteService {

```

```

    // Repositorio de estudiantes que maneja el almacenamiento y
    acceso a los datos

    private final EstudianteRepository repo;

    /**
     * Constructor que recibe el repositorio a utilizar.
     * @param repo el repositorio de estudiantes
     */
    public EstudianteServiceBase(EstudianteRepository repo) {
        this.repo = repo;
    }

    /**
     * Agrega un nuevo estudiante si no existe otro con el mismo ID.
     * @param id ID del estudiante
     * @param nombre Nombre del estudiante
     * @param edad Edad del estudiante
     */
    public void agregarEstudiante(int id, String nombre, int edad) {
        // Verifica si ya existe un estudiante con el mismo ID
        if (repo.buscarPorId(id) != null) {
            throw new IllegalArgumentException("El estudiante ya
existe.");
        }

        // Si no existe, lo agrega al repositorio
        repo.agregar(new Estudiante(id, nombre, edad));
    }

    /**
     * Busca un estudiante por su ID.
     * @param id ID del estudiante a buscar
     * @return el objeto Estudiante si existe, o null si no
     */
    public Estudiante buscarEstudiante(int id) {

```

```

        return repo.buscarPorId(id);
    }

    /**
     * Obtiene una lista con todos los estudiantes almacenados.
     * @return lista de estudiantes
     */
    public List<Estudiante> obtenerTodos() {
        return repo.obtenerTodos();
    }

    /**
     * Actualiza los datos de un estudiante existente.
     * @param id ID del estudiante a actualizar
     * @param nombre Nuevo nombre
     * @param edad Nueva edad
     */
    public void actualizarEstudiante(int id, String nombre, int
edad) {
        // Busca el estudiante por ID
        Estudiante est = repo.buscarPorId(id);

        // Si no existe, lanza excepción
        if (est == null) throw new IllegalArgumentException("No
existe el estudiante.");

        // Actualiza nombre y edad
        est.setNombre(nombre);

        est.setEdad(edad);

        // Guarda los cambios en el repositorio
        repo.actualizar(est);
    }

    /**
     * Elimina un estudiante por su ID.
     * @param id ID del estudiante a eliminar

```

```

    */

    public void eliminarEstudiante(int id) {

        repo.eliminar(id);

    }

}

```

Decorador logica_negocio (EstudianteServiceDecorador.java)

```

import datos.Model.Estudiante;
import java.util.List;

/**
 * Clase abstracta que actúa como decorador base para la interfaz
 * IEstudianteService.
 *
 * Implementa todos los métodos de la interfaz delegando las
 * llamadas al servicio decorado.
 *
 * Esta clase permite extender la funcionalidad del servicio
 * original sin modificar su código.
 */

public abstract class EstudianteServiceDecorator implements
IEstudianteService {

    // Referencia al servicio que se está decorando (puede ser
    EstudianteServiceBase o cualquier otro decorador)

    protected final IEstudianteService decorated;

    /**
     * Constructor que recibe el servicio a decorar.
     *
     * @param decorated implementación concreta de
     IEstudianteService
     */

    public EstudianteServiceDecorator(IEstudianteService decorated)
    {

        this.decorated = decorated;

    }

    /**

```



```
    * Método para agregar un estudiante.
    * Delegado directamente al servicio decorado.
    */
    public void agregarEstudiante(int id, String nombre, int edad) {
        decorated.agregarEstudiante(id, nombre, edad);
    }

    /**
     * Método para buscar un estudiante por ID.
     * Delegado directamente al servicio decorado.
     */
    public Estudiante buscarEstudiante(int id) {
        return decorated.buscarEstudiante(id);
    }

    /**
     * Método para obtener todos los estudiantes.
     * Delegado directamente al servicio decorado.
     */
    public List<Estudiante> obtenerTodos() {
        return decorated.obtenerTodos();
    }

    /**
     * Método para actualizar los datos de un estudiante.
     * Delegado directamente al servicio decorado.
     */
    public void actualizarEstudiante(int id, String nombre, int
edad) {
        decorated.actualizarEstudiante(id, nombre, edad);
    }

    /**
     * Método para eliminar un estudiante.
```

```

        * Delegado directamente al servicio decorado.
    */

    public void eliminarEstudiante(int id) {
        decorated.eliminarEstudiante(id);
    }
}

```

Decorador logica_negocio (EstudianteService.java)

```

import datos.Model.Estudiante;
import java.util.List;

/**
 * Interfaz que define el contrato para la gestión de estudiantes.
 * Esta interfaz permite abstraer la lógica de negocio del manejo de
 * estudiantes
 * y facilita el uso del patrón de diseño Decorator, ya que
 * múltiples clases pueden implementarla.
 */
public interface IEstudianteService {

    /**
     * Agrega un nuevo estudiante.
     * @param id ID del estudiante
     * @param nombre Nombre del estudiante
     * @param edad Edad del estudiante
     */
    void agregarEstudiante(int id, String nombre, int edad);

    /**
     * Busca un estudiante por su ID.
     * @param id ID del estudiante a buscar
     * @return el estudiante encontrado o null si no existe
     */
    Estudiante buscarEstudiante(int id);
}

```

```

    /**
     * Obtiene una lista con todos los estudiantes registrados.
     * @return lista de estudiantes
     */
    List<Estudiante> obtenerTodos();

    /**
     * Actualiza los datos de un estudiante existente.
     * @param id ID del estudiante a actualizar
     * @param nombre Nuevo nombre
     * @param edad Nueva edad
     */
    void actualizarEstudiante(int id, String nombre, int edad);

    /**
     * Elimina un estudiante por su ID.
     * @param id ID del estudiante a eliminar
     */
    void eliminarEstudiante(int id);
}

```

Decorador presentacion (EstudianteUI.java)

```

import logica_negocio.IEstudianteService;
import datos.Model.Estudiante;

import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.ActionEvent;
import javax.swing.event.DocumentEvent;
import javax.swing.event.DocumentListener;

```

```

/**
 * Ventana principal de la aplicación CRUD de estudiantes.
 * Esta clase representa la capa de presentación usando Swing.
 * Permite al usuario agregar, mostrar, buscar, actualizar y
eliminar estudiantes,
 * mostrando los datos en una tabla organizada.
 */
public class EstudianteUI extends JFrame {

    // Referencia al servicio de lógica de negocio
    private final IEstudianteService service;

    // Campos de texto para entrada de datos
    private final JTextField campoId = new JTextField(5);
    private final JTextField campoNombre = new JTextField(15);
    private final JTextField campoEdad = new JTextField(5);

    // Modelo y tabla para mostrar los estudiantes de forma
organizada
    private final DefaultTableModel modeloTabla = new
DefaultTableModel(
        new Object[]{"ID", "Nombre", "Edad"}, 0
    );

    private final JTable tabla = new JTable(modeloTabla);

    /**
     * Constructor que configura la interfaz gráfica,
     * organiza los paneles y conecta los botones con sus acciones.
     */
    public EstudianteUI(IEstudianteService service) {

        this.service = service;

        setTitle("CRUD Estudiantes");

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        setLayout(new BorderLayout());

        // Panel de entrada de datos
        JPanel panelEntrada = new JPanel();

```

```
panelEntrada.add(new JLabel("ID:"));

panelEntrada.add(campoId);

panelEntrada.add(new JLabel("Nombre:"));

panelEntrada.add(campoNombre);

panelEntrada.add(new JLabel("Edad:"));

panelEntrada.add(campoEdad);


// Panel de botones para las operaciones CRUD
JPanel panelBotones = new JPanel();

JButton btnAgregar = new JButton("Agregar");
JButton btnMostrar = new JButton("Mostrar Todos");
JButton btnActualizar = new JButton("Actualizar");
JButton btnEliminar = new JButton("Eliminar");


panelBotones.add(btnAgregar);
panelBotones.add(btnMostrar);
panelBotones.add(btnActualizar);
panelBotones.add(btnEliminar);


// Área de tabla para mostrar los estudiantes
JScrollPane scroll = new JScrollPane(tabla);


add(panelEntrada, BorderLayout.NORTH);
add(panelBotones, BorderLayout.CENTER);
add(scroll, BorderLayout.SOUTH);


// Asignación de acciones a los botones
btnAgregar.addActionListener(this::agregar);
btnMostrar.addActionListener(e -> mostrarTodos());
btnActualizar.addActionListener(this::actualizar);
btnEliminar.addActionListener(this::eliminar);


// Búsqueda en tiempo real al escribir en el campo de ID
```

```

        campoId.getDocument().addDocumentListener(new
DocumentListener() {

            public void insertUpdate(DocumentEvent e) {
buscarEnTiempoReal(); }

            public void removeUpdate(DocumentEvent e) {
buscarEnTiempoReal(); }

            public void changedUpdate(DocumentEvent e) {
buscarEnTiempoReal(); }

        });

        pack();

        setLocationRelativeTo(null);

        setVisible(true);

    }

    // Métodos privados para manejar cada acción CRUD y actualizar
la tabla

    private void agregar(ActionEvent e) {

        try {

            int id = Integer.parseInt(campoId.getText());

            String nombre = campoNombre.getText();

            int edad = Integer.parseInt(campoEdad.getText());

            service.agregarEstudiante(id, nombre, edad);

            mostrarTodos();

            JOptionPane.showMessageDialog(this, "Estudiante
agregado.");

        } catch (Exception ex) {

            JOptionPane.showMessageDialog(this, "Error: " +
ex.getMessage());

        }

    }

    private void mostrarTodos() {

        modeloTabla.setRowCount(0); // Limpiar tabla

        for (Estudiante est : service.obtenerTodos()) {

```

```

        modeloTabla.addRow(new Object[]{est.getId(),
est.getNombre(), est.getEdad()});

    }

}

// Búsqueda en tiempo real según lo que se escriba en el campo
de ID

private void buscarEnTiempoReal() {

    String textoId = campoId.getText();

    if (textoId.isEmpty()) {

        mostrarTodos();

        return;

    }

    try {

        int id = Integer.parseInt(textoId);

        Estudiante est = service.buscarEstudiante(id);

        modeloTabla.setRowCount(0);

        if (est != null) {

            modeloTabla.addRow(new Object[]{est.getId(),
est.getNombre(), est.getEdad()});

        }

    } catch (NumberFormatException ex) {

        modeloTabla.setRowCount(0); // Si no es un número,
limpia la tabla

    }

}

private void actualizar(ActionEvent e) {

    try {

        int id = Integer.parseInt(campoId.getText());

        String nombre = campoNombre.getText();

        int edad = Integer.parseInt(campoEdad.getText());

        service.actualizarEstudiante(id, nombre, edad);

        mostrarTodos();

    }

}

```

```

        JOptionPane.showMessageDialog(this, "Estudiante
actualizado.");
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this, "Error: " +
ex.getMessage());
    }
}

private void eliminar(ActionEvent e) {
    try {
        int id = Integer.parseInt(campoId.getText());
        service.eliminarEstudiante(id);
        mostrarTodos();
        JOptionPane.showMessageDialog(this, "Estudiante
eliminado.");
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this, "Error: " +
ex.getMessage());
    }
}
}

```

Características

Patrón de diseño Decorator	
1	Permite añadir funcionalidades adicionales (como validación o logging) a la lógica de negocio base sin modificar su código original, siguiendo el principio de abierto/cerrado
2	Cada decorador envuelve al servicio base o a otro decorador, creando una cadena de responsabilidades donde cada capa añade su comportamiento específico.
3	Las funcionalidades pueden ser combinadas de diferentes maneras en tiempo de ejecución, proporcionando gran

	flexibilidad.
4	Facilita el mantenimiento ya que cada funcionalidad adicional está encapsulada en su propia clase, permitiendo modificaciones independientes.

2. Iterator

Estructura del Proyecto

```

├─ Main.java
├─ datos/
│   └─ model/Estudiante.java
│   └─ repository/EstudianteRepository.java
├─ logica_negocio/
│   └─ EstudianteService.java
├─ presentacion/
│   └─ EstudianteUI.java

```

Código

Clase Main.java

```

import presentación.EstudianteUI;

/**
 * Clase Main
 *
 * Punto de entrada de la aplicación.
 * Su función principal es iniciar la interfaz gráfica de usuario
 * (GUI) para la gestión de estudiantes.

```

```

    * Utiliza SwingUtilities para asegurar que la interfaz se ejecute
    en el hilo de eventos de Swing,

    * lo cual es una buena práctica en aplicaciones Java con Swing.

    */
public class Main {

    public static void main(String[] args) {

        // Lanza la ventana principal de la aplicación en el hilo
        de eventos de Swing

        javax.swing.SwingUtilities.invokeLater(() -> new
        EstudianteUI());

    }

}

```

Estudiante.java:

```

package datos.Model;

/**
 * Clase Estudiante
 *
 * Representa el modelo de datos para un estudiante, encapsulando
    sus atributos principales:
    * id, nombre y edad. Esta clase se utiliza para crear objetos que
    almacenan y transfieren
    * la información de los estudiantes dentro de la aplicación.
 */
public class Estudiante {

    // Atributos que identifican y describen al estudiante

    private int id;

    private String nombre;

```

```

private int edad;

/**
 * Constructor principal.
 * Permite crear un estudiante con todos sus datos.
 */
public Estudiante(int id, String nombre, int edad) {
    this.id = id;
    this.nombre = nombre;
    this.edad = edad;
}

// Métodos de acceso (getters y setters) para los atributos del
estudiante

public int getId() { return id; }
public void setId(int id) { this.id = id; }
public String getNombre() { return nombre; }
public void setNombre(String nombre) { this.nombre = nombre; }
public int getEdad() { return edad; }
public void setEdad(int edad) { this.edad = edad; }

/**
 * Devuelve una representación en texto del estudiante.
 * Útil para mostrar información en la interfaz o para
depuración.
 */
@Override
public String toString() {
    return id + " - " + nombre + " (" + edad + " años)";
}

```

```
}
```

EstudianteRepository.java:

```
package datos.Repository;

import datos.Model.Estudiante;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

/**
 * Clase EstudianteRepository
 *
 * Esta clase funciona como un repositorio en memoria para objetos
Estudiante.
 *
 * Permite almacenar, buscar, eliminar y recorrer estudiantes,
simulando una base de datos simple.
 *
 * Es utilizada por la lógica de negocio para gestionar la
colección de estudiantes.
 */
public class EstudianteRepository {

    // Lista interna que almacena los estudiantes en memoria
    private List<Estudiante> repositorio = new ArrayList<>();

    /**
     * Agrega un nuevo estudiante al repositorio.
     */
    public void agregar(Estudiante e) { repositorio.add(e); }
```

```

/**
 * Elimina un estudiante por su ID.
 *
 * Utiliza un Iterator para evitar problemas al eliminar
durante la iteración.
 *
 * @return true si se eliminó, false si no se encontró.
 */

public boolean eliminar(int id) {

    Iterator<Estudiante> it = repositorio.iterator();

    while (it.hasNext()) {

        if (it.next().getId() == id) {

            it.remove();

            return true;

        }

    }

    return false;

}

/**
 * Busca y retorna un estudiante por su ID.
 *
 * @return El estudiante si existe, o null si no se encuentra.
 */

public Estudiante buscar(int id) {

    for (Estudiante e : repositorio)

        if (e.getId() == id) return e;

    return null;

}

/**
 * Devuelve un Iterator para recorrer todos los estudiantes
almacenados.

```

```

        * Útil para patrones de diseño como Iterator o para recorrer
la colección externamente.

        */

    public Iterator<Estudiante> obtenerIterator() {

        return repositorio.iterator();

    }

}

```

EstudianteService.java:

```

package lógica_negocio;

import datos.Model.Estudiante;
import datos.Repository.EstudianteRepository;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

/**
 * Clase EstudianteService
 *
 * Esta clase representa la capa de lógica de negocio para la
gestión de estudiantes.
 * Se encarga de coordinar las operaciones CRUD (crear, leer,
actualizar, eliminar)
 * utilizando el repositorio de estudiantes como fuente de datos.
 * Sirve de intermediario entre la interfaz de usuario y la capa de
datos.
 */

public class EstudianteService {

```

```
// Instancia del repositorio que almacena los estudiantes

private EstudianteRepository repo = new EstudianteRepository();

/**
 * Crea y agrega un nuevo estudiante al repositorio.
 */
public void crear(int id, String nombre, int edad) {
    repo.agregar(new Estudiante(id, nombre, edad));
}

/**
 * Busca y retorna un estudiante por su ID.
 */
public Estudiante leer(int id) {
    return repo.buscar(id);
}

/**
 * Actualiza los datos de un estudiante existente.
 * @return true si se actualizó, false si no se encontró el
estudiante.
 */
public boolean actualizar(int id, String nombre, int edad) {
    Estudiante e = repo.buscar(id);

    if (e != null) {
        e.setNombre(nombre);
        e.setEdad(edad);
        return true;
    }
}
```

```

        return false;
    }

    /**
     * Elimina un estudiante por su ID.
     * @return true si se eliminó, false si no se encontró.
     */
    public boolean eliminar(int id) {
        return repo.eliminar(id);
    }

    /**
     * Devuelve una lista con todos los estudiantes almacenados.
     * Utiliza un Iterator para recorrer el repositorio y construir
    la lista.
     */
    public List<Estudiante> listarTodos() {
        List<Estudiante> lista = new ArrayList<>();
        Iterator<Estudiante> it = repo.obtenerIterator();
        while (it.hasNext()) {
            lista.add(it.next());
        }
        return lista;
    }
}

```

EstudianteUI.java:

```
package presentación;
```



```
import lógica_negocio.EstudianteService;

import datos.Model.Estudiante;


import javax.swing.*;

import javax.swing.table.DefaultTableModel;

import java.awt.*;

import java.awt.event.ActionEvent;

import javax.swing.event.DocumentEvent;

import javax.swing.event.DocumentListener;


/**
 * Ventana principal para la gestión de estudiantes.
 *
 * Esta clase representa la interfaz gráfica de usuario (GUI) de la
aplicación.
 *
 * Permite al usuario crear, buscar, actualizar y eliminar
estudiantes,
 *
 * mostrando los datos en una tabla. Sirve como puente entre el
usuario y la lógica de negocio.
 */
public class EstudianteUI extends JFrame {

    // Servicio de lógica de negocio para gestionar estudiantes
    private final EstudianteService service = new
EstudianteService();


    // Campos de texto para ingresar datos del estudiante
    private final JTextField txtId = new JTextField(5);

    private final JTextField txtNombre = new JTextField(15);

    private final JTextField txtEdad = new JTextField(5);
```

```

        // Modelo y tabla para mostrar los estudiantes en la interfaz

        private final DefaultTableModel modeloTabla = new
DefaultTableModel(

            new Object[]{"ID", "Nombre", "Edad"}, 0

        );

        private final JTable tabla = new JTable(modeloTabla);

        /**

            * Constructor que configura la ventana, los paneles y los
eventos.

            * Organiza la interfaz en paneles para entrada de datos,
botones y la tabla.

            * Conecta los botones y campos con sus respectivas acciones.

            */

        public EstudianteUI() {

            setTitle("Gestión de Estudiantes");

            setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

            setLayout(new BorderLayout(10, 10));

            // Panel de entrada de datos (arriba)

            JPanel panelEntrada = new JPanel();

            panelEntrada.add(new JLabel("ID:"));

            panelEntrada.add(txtId);

            panelEntrada.add(new JLabel("Nombre:"));

            panelEntrada.add(txtNombre);

            panelEntrada.add(new JLabel("Edad:"));

            panelEntrada.add(txtEdad);

            // Panel de botones (centro)

```

```
JPanel panelBotones = new JPanel();

JButton btnCrear = new JButton("Crear");

JButton btnMostrar = new JButton("Mostrar Todos");

JButton btnActualizar = new JButton("Actualizar");

JButton btnEliminar = new JButton("Eliminar");


panelBotones.add(btnCrear);

panelBotones.add(btnMostrar);

panelBotones.add(btnActualizar);

panelBotones.add(btnEliminar);


// Tabla para mostrar estudiantes (abajo)
JScrollPane scroll = new JScrollPane(tabla);


add(panelEntrada, BorderLayout.NORTH);

add(panelBotones, BorderLayout.CENTER);

add(scroll, BorderLayout.SOUTH);


// Asignación de acciones a los botones

btnCrear.addActionListener(this::crear);

btnMostrar.addActionListener(_ -> mostrarTodos());

btnActualizar.addActionListener(this::actualizar);

btnEliminar.addActionListener(this::eliminar);


// Búsqueda en tiempo real por ID al escribir en el campo
correspondiente

txtId.getDocument().addDocumentListener(new
DocumentListener() {

    public void insertUpdate(DocumentEvent e) {
        buscarEnTiempoReal();
    }
}
```

```

        public void removeUpdate(DocumentEvent e) {
buscarEnTiempoReal(); }

        public void changedUpdate(DocumentEvent e) {
buscarEnTiempoReal(); }

    });

    pack();

    setLocationRelativeTo(null);

    setVisible(true);

    mostrarTodos();

}

/**
 * Crea un nuevo estudiante con los datos ingresados y lo
agrega al sistema.
 * Muestra un mensaje de confirmación o error.
 */
private void crear(ActionEvent e) {

    try {

        int id = Integer.parseInt(txtId.getText());

        String nombre = txtNombre.getText();

        int edad = Integer.parseInt(txtEdad.getText());

        service.crear(id, nombre, edad);

        mostrarTodos();

        JOptionPane.showMessageDialog(this, "Estudiante
creado.");

    } catch (Exception ex) {

        JOptionPane.showMessageDialog(this, "Error: " +
ex.getMessage());

    }

}

```

```
/**
 * Muestra todos los estudiantes en la tabla.
 * Limpia la tabla y la llena con los datos actuales.
 */

private void mostrarTodos() {

    modeloTabla.setRowCount(0);

    for (Estudiante est : service.listarTodos()) {

        modeloTabla.addRow(new Object[]{est.getId(),
est.getNombre(), est.getEdad()});

    }

}

/**
 * Busca un estudiante por ID en tiempo real mientras se
escribe.
 * Si el campo está vacío, muestra todos los estudiantes.
 * Si el ID es válido y existe, muestra solo ese estudiante.
 */

private void buscarEnTiempoReal() {

    String textoId = txtId.getText();

    if (textoId.isEmpty()) {

        mostrarTodos();

        return;

    }

    try {

        int id = Integer.parseInt(textoId);

        Estudiante est = service.leer(id);

        modeloTabla.setRowCount(0);

        if (est != null) {
```

```

        modeloTabla.addRow(new Object[]{est.getId(),
est.getNombre(), est.getEdad()});

    }

    } catch (NumberFormatException ex) {

        modeloTabla.setRowCount(0);

    }

}

/**
 * Actualiza los datos de un estudiante existente.
 * Muestra un mensaje de confirmación o error.
 */
private void actualizar(ActionEvent e) {

    try {

        int id = Integer.parseInt(txtId.getText());

        String nombre = txtNombre.getText();

        int edad = Integer.parseInt(txtEdad.getText());

        boolean ok = service.actualizar(id, nombre, edad);

        mostrarTodos();

        JOptionPane.showMessageDialog(this, ok ? "Estudiante
actualizado." : "No encontrado.");

    } catch (Exception ex) {

        JOptionPane.showMessageDialog(this, "Error: " +
ex.getMessage());

    }

}

/**
 * Elimina un estudiante por ID.
 * Muestra un mensaje de confirmación o error.

```

```

*/

private void eliminar(ActionEvent e) {

    try {

        int id = Integer.parseInt(txtId.getText());

        boolean ok = service.eliminar(id);

        mostrarTodos();

        JOptionPane.showMessageDialog(this, ok ? "Estudiante
eliminado." : "No encontrado.");

    } catch (Exception ex) {

        JOptionPane.showMessageDialog(this, "Error: " +
ex.getMessage());

    }

}

}

```

Características

Patrón de diseño Iterator	
1	Proporciona una forma de acceder secuencialmente a los elementos de un objeto agregado sin exponer su estructura interna.
2	Separa la responsabilidad de iterar de la colección misma, promoviendo el principio de responsabilidad única.
3	Permite recorrer una colección sin conocer su implementación interna (ArrayList, LinkedList, etc.).
4	Facilita operaciones seguras durante la iteración, como la eliminación de elementos sin errores de concurrencia.

Conclusiones

- El proyecto sigue una arquitectura organizada con capas bien definidas: presentación, lógica de negocio, repositorio y modelo. Esto facilita el mantenimiento, escalabilidad y pruebas.
- La implementación del patrón Decorator permite extender las funcionalidades del sistema de manera modular y dinámica, manteniendo el código base intacto y cumpliendo con el principio de abierto/cerrado. Este enfoque facilita la adición de nuevas características (como validaciones, logging o notificaciones) sin modificar el núcleo existente, mejorando la mantenibilidad y escalabilidad del sistema.
- El patrón Iterator es fundamental para recorrer colecciones de objetos de manera eficiente y desacoplada, ocultando la estructura interna de los datos y permitiendo múltiples formas de iteración (secuencial, filtrada, etc.) . Su implementación promueve el principio de responsabilidad única y facilita la extensión del código para nuevas estructuras de datos sin modificar el cliente existente.

Recomendaciones

- **Definir interfaces claras:** Asegurar que tanto el componente base como los decoradores implementen una interfaz común para garantizar la compatibilidad y flexibilidad al combinar funcionalidades.
- **Evitar anidamientos excesivos:** Limitar la cantidad de decoradores anidados para no complicar el flujo del sistema. Priorizar composiciones simples y documentar las combinaciones más usadas.

- **Encapsular el acceso al Iterator:** Aunque se expone directamente el Iterator desde el repositorio, se recomienda encapsular más el acceso o proporcionar una interfaz más específica si en el futuro se desea controlar el comportamiento de iteración (e.g., filtros personalizados).
 - **Agregar soporte para iteraciones más complejas:** Puedes extender la funcionalidad para iteraciones por condiciones (e.g., estudiantes mayores de cierta edad), implementando iteradores personalizados si el dominio lo requiere.
-

3. Referencias (Norma APA 7.0)

- Descargar IntelliJ IDEA (2021, June 1). JetBrains.
<https://www.jetbrains.com/es-es/idea/download/download-thanks.html?platform=windows&code=IIC>
- Leiva, A. (n.d.). Patrones de diseño de software | DevExpert.
<https://devexpert.io/blog/patrones-de-diseno-software>
- Iterator. (n.d.). <https://refactoring.guru/es/design-patterns/iterator>
- Iterator en Java / Patrones de diseño. (n.d.).
<https://refactoring.guru/es/design-patterns/iterator/java/example>
- Admin. (2023, September 16). Patrón Iterator en PHP: Recorriendo Colecciones de Objetos sin Exponer su Estructura Interna. Guia PHP.
<https://guiaphp.com/desarrollo/patron-iterator-en-php-recorriendo-colecciones-de-objetos-sin-exponer-su-estructura-interna/>
- Kumar, S. (2024, November 19). Day 18: Iterator pattern — Accessing elements without exposing the underlying structure. Medium.
<https://sourabhkr.medium.com/day-18-iterator-pattern-accessing-elements-without-exposing-the-underlying-structure-24617aaff868>
- Dhanjal, S. (2022, May 26). Iterator Design Pattern - Scaler topics. Scaler Topics.
<https://www.scaler.com/topics/design-patterns/iterator-design-pattern/>